ENGINEERING TECHNICAL BRIEF

# Enterprise Workspace

React / Express / PostgreSQL / Prisma ORM / SQL workspace

## PROJECT OVERVIEW

A local business-data workspace for relational tables, record editing, schema inspection and ad hoc SQL. The frontend is a React/Vite application; an Express API uses Prisma and PostgreSQL to store core entities, custom table definitions and actual records.

## APPLICATION FLOW

### React interface

Customizable workspace, dashboard, table editor, relationships, query console and query builder.

### Express API

CRUD endpoints, schema inspection, dynamic SQL helpers and query execution.

### PostgreSQL

Core relational entities, physical custom tables, foreign keys and workspace metadata.

## RELATIONAL FOUNDATION

### Product

sku (unique), price DECIMAL(10,2), stock, categoryId and supplierId foreign keys.

### Category

Unique name and one-to-many relation to Product.

### Supplier

Unique email and one-to-many relation to Product.

## DYNAMIC TABLE DEFINITIONS

WorkspaceTable and WorkspaceColumn hold custom table/column metadata including position, type, primary-key and uniqueness flags, nullability, defaults, auto-increment and optional referenced table/column IDs. The API builds CREATE TABLE / ALTER TABLE SQL and operates on physical PostgreSQL tables, rather than only adding columns in the browser.

## SCHEMA INSPECTION

The schema endpoint reads PostgreSQL information_schema for columns, keys and relationships, and counts physical table records. React uses this data for the dashboard, schema view and relationship diagrams.

ENGINEERING TECHNICAL BRIEF

# Implementation and limits

How the editing, SQL and browser-state features are represented in source.

## WORKSPACE OPERATIONS

### Table and record editing

The workspace API creates, renames and removes custom tables, edits columns and records, validates supported types, and handles foreign-key definitions. Prisma models the metadata; generated SQL changes physical tables.

### Queries and inspection

The query console executes submitted SQL and returns rows, columns and counts. The browser query builder constructs SELECT statements with selected columns, a condition, sort direction and optional LIMIT.

## PERSISTENCE AND USER STATE

Products, categories, suppliers, workspace definitions and records persist in PostgreSQL. Workspace page selection, query-builder state and query history are stored in localStorage, while the demonstration login flag is kept in sessionStorage. The browser state is not synchronized to authenticated accounts.

## IMPLEMENTATION BOUNDARIES

### Demo login

The client checks fixed demo credentials and stores a session flag. There is no server-side identity or role-based access-control layer protecting the API routes.

### Unrestricted query endpoint

POST /api/query forwards a user-supplied SQL string to prisma.$queryRawUnsafe. No read-only enforcement or least-privilege query role is shown. This must not be exposed to untrusted users.

### Schema consistency

Custom metadata and physical tables must remain consistent through DDL changes. Tests for interrupted migrations, partial failures and rollback behavior are needed before production use.

### Validation status

Inspected source includes Prisma migrations, Express controllers and React pages. No committed automated test suite, CI workflow or benchmark evidence was found during this review.

## REFERENCE FILES

server/prisma/schema.prisma
server/src/controllers/workspaceTableController.js
server/src/controllers/queryController.js
server/src/utils/dynamicTable.js
client/src/pages/QueryBuilder/QueryBuilder.jsx
client/src/pages/Relationships/Relationships.jsx